

REILO

Referral Intelligence Platform

System Architecture Design Document

Version 1.0 · Confidential

Prepared: June 2026

Stack at a Glance

Layer	Technology
Auth & OAuth	Firebase Authentication (Google Sign-In, Email/Password)
Backend API	Node.js + Express (REST) hosted on Railway (free tier)
Database	MongoDB Atlas (free M0 cluster)
AI / Email Gen	Google Gemini API (gemini-2.0-flash, free tier)
Email Delivery	Resend (free 3,000 emails/mo) or Brevo (free 300/day)
Frontend	React Native (Expo) — iOS & Android from one codebase
Push Notifications	Firebase Cloud Messaging (FCM) — free
File / Asset CDN	Cloudinary (free tier) for avatars & branding assets
CI / CD	GitHub Actions (free for public / private repos)

1. Executive Summary

Reilo is a mobile-first referral intelligence platform that enables professionals to manage their referral network, track job referrals, and automate referral outreach — all at zero cost. The architecture is deliberately built on free, open-source-friendly services that can scale to early product-market-fit without any cloud spend.

This document covers:

- High-level system architecture and component responsibilities
- Data models for all primary entities
- Authentication and authorisation design
- Feature flows for all three application modules
- AI integration strategy (Gemini API for email generation)
- Email delivery pipeline with branding
- Notification system design (FCM)
- Security threat model and mitigations
- Free-tier cost map — every dependency annotated with its free limit

2. High-Level System Architecture

The architecture follows a classic three-tier pattern: a React Native mobile client, a stateless Node.js REST API on Railway, and a MongoDB Atlas data layer. Firebase acts exclusively as the identity provider and push notification bus — all business data lives in MongoDB.

2.1 Architecture Layers

Layer	Component	Responsibility
Presentation	React Native / Expo	Mobile UI, state management (Zustand), JWT storage (Expo SecureStore), deep-link handling
Identity	Firebase Auth	Credential verification, ID token issuance, refresh token rotation, OAuth providers
API Gateway	Express Middleware	JWT verification, rate limiting, CORS, request validation (Zod)
Business Logic	Node.js / Express	Referral management, job tracking, email composition, AI orchestration, FCM dispatch
AI Engine	Gemini 2.0 Flash	Context-aware referral email generation, subject line suggestion
Persistence	MongoDB Atlas	Users, referrals, jobs, email threads, notification logs
Email Delivery	Resend / Brevo	Transactional branded email with reply-to webhook support
Notifications	Firebase FCM	Push notifications for referral accept/decline, job status updates
Media / Assets	Cloudinary	Reilo branding assets, user avatars (optimised CDN delivery)

2.2 Component Interaction Diagram (Textual)

The following describes the runtime request flow:

1. User opens Reilo → Expo SecureStore checks for a stored Firebase ID token
2. If no token or token expired → Firebase onAuthStateChanged triggers the auth flow (see Section 4)
3. Authenticated requests: client attaches the Firebase ID token as a Bearer token in the Authorization header
4. Express middleware calls firebase-admin.auth().verifyIdToken() to validate the JWT cryptographically
5. Validated uid is injected into req.user; business logic uses this as the authoritative user identifier
6. MongoDB queries are always scoped to the authenticated uid — no user can read another user's data
7. Outbound emails are dispatched via Resend SDK from the backend (never from the client)
8. AI email generation calls Gemini API server-side with a structured prompt; the client never holds the Gemini API key
9. Push notifications are dispatched via firebase-admin.messaging() from the backend

 The mobile client never holds the Gemini API key or the MongoDB connection string. Both are backend secrets stored as Railway environment variables.

2.3 Deployment Topology

Service	Host / Platform
React Native app	Expo EAS Build → App Store / Play Store
Node.js API	Railway (free starter plan — 500 hrs/mo, 512 MB RAM)
MongoDB	MongoDB Atlas M0 free cluster (512 MB, shared)
Firebase	Google Firebase free Spark plan
Email	Resend free (3,000 emails/mo) or Brevo (300/day)
Cloudinary	Cloudinary free (25 GB storage, 25 GB bandwidth)
CI / CD	GitHub Actions free (2,000 min/mo for private repos)

3. Data Models (MongoDB Collections)

3.1 users

 Created on first login via Firebase. uid is the Firebase UID and serves as the primary join key across all collections.

Field	Type	Notes
_id	ObjectId	MongoDB primary key
uid	String (unique)	Firebase UID — indexed, used as FK in all collections
email	String	Verified via Firebase
displayName	String	Full name from profile
profession	String	Job title / role
avatarUrl	String	Cloudinary CDN URL
fcmToken	String	Latest FCM device token (updated on each login)
createdAt	Date	ISO timestamp
updatedAt	Date	Auto-managed via Mongoose timestamps

3.2 referrals

💡 Each referral belongs to exactly one user (ownerUid). A referral represents a contact who may refer the user for jobs.

Field	Type	Notes
_id	ObjectId	Primary key
ownerUid	String	FK → users.uid (indexed)
name	String	Full name of the referral contact
company	String	Current employer
companyId	String	Internal/ATS company identifier (optional)
email	String	Used for outreach emails
linkedinUrl	String	Profile URL
phoneNumber	String	Optional
consentStatus	Enum	pending accepted declined
consentSentAt	Date	Timestamp of consent invite email
consentRespondedAt	Date	Timestamp of consent response
jobsReferred	ObjectId[]	Array of FK → jobs._id
createdAt	Date	
updatedAt	Date	

3.3 jobs

💡 A job is a specific role the user is targeting for a referral. A job can be linked to multiple referrals.

Field	Type	Notes
_id	ObjectId	Primary key
ownerUid	String	FK → users.uid (indexed)
jobId	String	External job board ID / ATS reference
jobRole	String	e.g. Senior Frontend Engineer
jobDescription	String	Full JD text — used as Gemini context
company	String	Hiring company
referralId	ObjectId	FK → referrals._id (the chosen referrer)
referralStatus	Enum	pending email_sent accepted declined
emailThreadId	ObjectId	FK → emailThreads._id
createdAt	Date	
updatedAt	Date	

3.4 emailThreads

💡 Stores the AI-generated (or human-edited) email draft plus delivery receipts. Kept separate from jobs for auditability.

Field	Type	Notes
_id	ObjectId	Primary key
ownerUid	String	FK → users.uid
jobId	ObjectId	FK → jobs._id
referralId	ObjectId	FK → referrals._id
subject	String	AI-suggested or human-written subject line
bodyAI	String	Raw Gemini-generated body
bodyFinal	String	User-edited body actually sent
sentAt	Date	Timestamp of send
resendMessageId	String	Provider message ID for tracking
referralResponse	Enum	null accepted declined
responseAt	Date	When referral clicked accept/decline

4. Authentication & Authorisation

4.1 Authentication Flow

Firebase Authentication handles all credential management. The backend never stores passwords.

10. Client calls Firebase SDK `signInWithEmailAndPassword()` or `signInWithCredential()` (Google OAuth)
11. Firebase returns an ID token (JWT, 1-hour expiry) and a refresh token
12. Client stores ID token in Expo SecureStore (hardware-backed Keychain / Keystore)
13. Every API call sends `Authorization: Bearer <idToken>`
14. Express `verifyToken` middleware calls `firebase-admin.auth().verifyIdToken(token)`
15. On success: `uid` is attached to `req.user.uid` and the request proceeds
16. On expiry: client SDK auto-refreshes using the refresh token and retries

4.2 Authorisation Model

All MongoDB queries are hard-scoped to `req.user.uid`. No RBAC is required for v1 — every user owns exactly their own data.

⚠ *Never trust the `userId` from the request body. Always derive it from the verified Firebase token (`req.user.uid`). This prevents horizontal privilege escalation.*

4.3 Consent Token (Referral Invite)

When a user adds a referral and triggers the consent email, the backend generates a signed consent token:

- Algorithm: HS256 JWT signed with a 256-bit secret stored as a Railway environment variable
- Payload: `{ referralId, ownerId, action: 'consent', exp: 7 days }`
- The token is embedded as a URL parameter in the consent email links (yes/no buttons)
- A dedicated public endpoint `POST /consent/:token` validates and processes the response without requiring authentication (the referral is an external party)
- Token is single-use — the referral document is updated and the token invalidated on first use

💡 *Consent tokens are stateless JWTs. Do not store them in MongoDB — rely on the expiry and single-use flag set on the referral document (`consentRespondedAt` not null = used).*

5. Feature Module Design

5.1 Module 1 — My Referrals

5.1.1 Referral List Screen

`GET /api/referrals` — returns all referrals where `ownerUid === req.user.uid`, including aggregated stats (total referrals, consent accepted count, jobs referred count). Pagination supported via cursor.

5.1.2 Add Referral Flow

17. User fills form: name, company, email, LinkedIn URL, phone, companyId
18. Client sends `POST /api/referrals` with form payload
19. Backend validates with Zod schema, creates referral document (`consentStatus: 'pending'`)
20. Backend calls Email Service to dispatch the Reilo-branded consent invite
21. Consent email contains two CTA buttons linking to `GET /consent/:token?action=yes` and `GET /consent/:token?action=no`

22. Referral clicks → backend validates consent token → updates consentStatus, consentRespondedAt
23. Backend dispatches FCM push to ownerId with consent outcome

5.1.3 Referral Detail Screen

GET /api/referrals/:referralId — returns full referral document with populated jobsReferred array. Each job card is tappable, navigating to the Job Description screen (Module 2).

5.1.4 Ask for Referral Flow

24. User taps Ask for Referral on a referral detail screen
25. User fills job detail form: jobId, jobRole, jobDescription
26. Client sends POST /api/jobs/draft with { jobDetails, referralId }
27. Backend creates a job document (referralStatus: 'pending') and immediately calls Gemini API
28. Gemini prompt includes: user's name, referral's name, company, job role, job description
29. Gemini returns JSON { subject: string, body: string } — stored as bodyAI in a new emailThread document
30. Client renders the AI draft in an editable text area (bodyFinal)
31. User reviews, edits, and taps Send
32. Client sends POST /api/jobs/:jobId/send-email with bodyFinal
33. Backend dispatches email via Resend; updates job.referralStatus to 'email_sent'
34. Referral's response link (accept/decline) in email updates job.referralStatus and triggers FCM to user

5.2 Module 2 — Referred Jobs

GET /api/jobs — returns all jobs owned by req.user.uid with referral status. Users can also initiate the full Add Job → Select Referral → AI Email → Send flow here directly (same backend endpoints as Module 1).

Job Description screen: GET /api/jobs/:jobId — returns full job + linked referral summary + email thread status.

5.3 Module 3 — Profile & Settings

The profile screen is intentionally minimal for v1. It includes:

- Display name and profession (editable → PATCH /api/users/me)
- Avatar upload (Cloudinary signed upload from client; URL stored in MongoDB)
- Email address (read-only — managed by Firebase)
- Change password (via Firebase SDK sendPasswordResetEmail — never handled by the backend)
- Push notification toggle (FCM token registration / deregistration)
- Sign out (Firebase SDK signOut + clear Expo SecureStore)
- Delete account (DELETE /api/users/me — cascades: referrals, jobs, emailThreads deleted; Firebase user deleted via admin SDK)

6. AI Email Generation (Gemini API)

6.1 Prompt Design

The backend constructs a structured system + user prompt for Gemini 2.0 Flash:

💡 System: You are a professional email writer for a referral request app called Reilo. Generate a concise, warm, and professional referral request email. Return ONLY valid JSON with keys: subject (string) and body (string). No markdown, no code fences.

User prompt variables injected:

- Sender's name and profession
- Referral's name and company
- Job role and job description (trimmed to 800 tokens to avoid quota overrun)
- Optional: user's brief personal note (passed in the request body)

6.2 Response Handling

The Gemini response is parsed as JSON. If parsing fails (Gemini occasionally returns markdown-wrapped JSON), the backend strips code fences with a regex and retries parsing once. If it still fails, the endpoint returns a 200 with a fallback template email and a flag `isAIGenerated: false` so the client can show an appropriate indicator.

6.3 Security

⚠️ The Gemini API key is stored exclusively as a Railway environment variable (`GEMINI_API_KEY`). It is never returned to the client, never logged, and never committed to the repository.

Rate limit applied on the `/api/jobs/draft` endpoint: 10 requests per user per hour (tracked in MongoDB with a TTL index to auto-expire rate limit documents).

7. Email Delivery Pipeline

7.1 Consent Invite Email

Triggered when a referral is added. Sent from `noreply@reilo.app` (custom domain on Resend).

- Subject: [First Name], [Owner Name] has added you as a referral on Reilo
- Body: Reilo-branded HTML with two large CTA buttons — Yes, I'll refer them and No, thanks
- Each CTA points to a tokenised backend URL (see Section 4.3)
- From name is displayed as Reilo · [Owner Name]

7.2 Referral Outreach Email

Sent when the user taps Send after reviewing the AI-generated draft.

- Subject and body from the `emailThread.bodyFinal` field
- From: [Owner Name] via Reilo <`noreply@reilo.app`>
- Reply-To: the user's actual email address (so replies land in their inbox naturally)
- Accept/Decline CTA buttons link to the same `/consent/:token` endpoint with an `action=accept` or `action=decline` parameter specific to the job

7.3 Provider Choice

Provider	Free Tier	SDK	Best For
Resend	3,000/month, 100/day	resend npm package	Developer-first, great React Email templates
Brevo	300/day unlimited months	brevo npm package	Higher daily volume on free tier

Recommendation: Start with Resend for its developer experience and React Email template support. The email service is abstracted behind a single `emailService.send()` utility function so the provider can be swapped without touching business logic.

8. Security Architecture

8.1 Threat Model

Threat	Attack Vector	Mitigation
Unauthorised data access	Attacker calls API with a crafted uid in the body	All queries use <code>req.user.uid</code> from verified Firebase token — body uid is ignored
Token theft	ID token stolen from device storage	Expo SecureStore (Keychain / Keystore) — hardware-backed, app-sandboxed
Consent token replay	Attacker reuses a captured consent URL	Single-use flag: <code>consentRespondedAt !== null</code> blocks re-use; 7-day expiry
JWT secret exposure	GEMINI_API_KEY or JWT_SECRET leaked	Railway env vars; never in codebase; <code>.env</code> in <code>.gitignore</code> ; GitHub secret scanning enabled
Mass email abuse	User sends thousands of AI emails	Rate limit: 10 /draft calls per user per hour (MongoDB TTL collection)
Injection attacks	Malicious <code>jobDescription</code> sent to Gemini	Zod input validation before any DB write or AI call; <code>jobDescription</code> truncated to 1000 chars
CORS misuse	Third-party site calls the API	Express CORS configured with explicit allowlist of Reilo origins only
Brute-force login	Attacker tries many passwords	Firebase Auth handles this natively (rate limiting, reCAPTCHA on web)
NoSQL injection	Malicious query operators in request	Mongoose ODM sanitises queries; <code>express-mongo-sanitize</code> middleware strips <code>\$</code> and <code>.</code> from inputs
Data leakage in logs	Email bodies or tokens logged to stdout	Structured logging (pino) with explicit field redaction for <code>email.body</code> , <code>token</code> , <code>authorization</code>

8.2 Security Headers

The Express API applies the following headers via helmet.js:

- Strict-Transport-Security: max-age=31536000 (HSTS)
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- Content-Security-Policy: default-src 'none' (API-only, no HTML served)
- Referrer-Policy: no-referrer

8.3 Secrets Management

All secrets are stored as Railway environment variables and are never committed to source control. The .env.example file documents required keys without values. GitHub repository is configured with secret scanning alerts enabled.

Required secrets:

- FIREBASE_SERVICE_ACCOUNT_JSON — Firebase Admin SDK credentials
- MONGO_URI — MongoDB Atlas connection string with least-privilege user
- GEMINI_API_KEY — Google AI Studio key
- JWT_SECRET — 256-bit random secret for consent tokens (openssl rand -hex 32)
- RESEND_API_KEY — Email delivery
- CLOUDINARY_URL — Cloudinary account credentials

9. REST API Design

All endpoints under /api/* require a valid Firebase ID token (Authorization: Bearer <token>) unless marked [PUBLIC].

9.1 Endpoint Reference

Method	Endpoint	Description
GET	/api/users/me	Get authenticated user's profile
PATCH	/api/users/me	Update display name, profession, avatarUrl, fcmToken
DELETE	/api/users/me	Delete account + cascade all data + Firebase user
GET	/api/referrals	List all referrals (with stats) for authenticated user
POST	/api/referrals	Create referral + dispatch consent email
GET	/api/referrals/:id	Get referral detail with populated jobs
PATCH	/api/referrals/:id	Update referral contact info
DELETE	/api/referrals/:id	Remove referral + unlink from jobs
GET	/api/jobs	List all jobs for authenticated user
POST	/api/jobs/draft	Create job + generate AI email draft (rate limited)
GET	/api/jobs/:id	Get job detail + email thread
POST	/api/jobs/:id/send-email	Finalise and send referral outreach email

Method	Endpoint	Description
PATCH	/api/jobs/:id	Update job details (before email is sent)
GET	/consent/:token [PUBLIC]	Process referral consent response (yes/no) — redirect to branded result page

10. Free Tier Cost Map

Reilo is engineered to operate entirely within free service tiers. The following table documents every dependency, its free allowance, and the expected ceiling before an upgrade would be needed.

Service	Free Allowance	Expected Usage (v1)	Upgrade Trigger
MongoDB Atlas M0	512 MB storage, shared compute	< 50 MB for 500 users	> 200 active users with large JDs
Firebase Auth	Unlimited (email/password + Google)	All auth traffic	Never on free — identity is always free
Firebase FCM	Unlimited push notifications	All push traffic	Never on free
Railway Starter	500 hrs/mo, 512 MB RAM, 1 GB egress	~350 hrs API uptime	> 500 hrs/mo or memory pressure
Gemini 2.0 Flash	15 RPM, 1M TPD free (AI Studio)	~50 emails/day	> 1M tokens/day
Resend	3,000 emails/mo, 100/day	~60 emails/day	> 3,000 emails/mo
Cloudinary	25 GB storage, 25 GB BW/mo	< 1 GB (avatars)	> 1,000 active users with avatars
GitHub Actions	2,000 min/mo (private repo)	~200 min/mo CI	> 2,000 min/mo
Expo EAS Build	30 builds/mo free	~10 builds/mo	> 30 builds/mo

💡 At approximately 500 monthly active users, MongoDB Atlas and Railway are the first services that will require upgrading. MongoDB Atlas Flex starts at ~\$9/month; Railway Pro is \$5/month. All other services remain free well beyond 1,000 users.

11. Recommended Repository Structure

11.1 Monorepo Layout

A single GitHub repository with two top-level packages simplifies CI/CD and keeps the project manageable.

Path	Contents
/apps/mobile/	Expo React Native app (screens, components, hooks, stores)

Path	Contents
/apps/mobile/src/screens/	ReferralListScreen, ReferralDetailScreen, AddReferralScreen, JobListScreen, JobDetailScreen, AskReferralScreen, EmailDraftScreen, ProfileScreen
/apps/api/	Node.js + Express backend
/apps/api/src/routes/	users.js, referrals.js, jobs.js, consent.js
/apps/api/src/services/	emailService.js, geminiService.js, fcmService.js, authService.js
/apps/api/src/models/	User.js, Referral.js, Job.js, EmailThread.js (Mongoose schemas)
/apps/api/src/middleware/	verifyToken.js, rateLimiter.js, validateRequest.js, sanitize.js
/apps/api/src/config/	firebase.js, db.js, cloudinary.js
/.github/workflows/	api-ci.yml, mobile-ci.yml
/docs/	This architecture document and ADRs

12. Architecture Decision Records (ADR Summary)

ADR-001: MongoDB over PostgreSQL

- Decision: Use MongoDB Atlas (free M0) instead of a relational database
- Rationale: Job descriptions and email bodies are variable-length text blobs that don't benefit from SQL normalisation. MongoDB's document model fits the referral → jobs → emailThread nesting naturally. Atlas M0 is free and managed.
- Trade-off: No joins (mitigated by Mongoose populate). No ACID multi-document transactions needed for v1.

ADR-002: Firebase Auth over custom JWT auth

- Decision: Delegate all authentication to Firebase Auth
- Rationale: Eliminates password hashing, token rotation, and OAuth implementation complexity. Firebase handles rate limiting and brute-force protection natively. Admin SDK provides server-side token verification in one line.
- Trade-off: Vendor dependency on Google. Mitigated by keeping all business data in MongoDB so migration would only require swapping the auth middleware.

ADR-003: Gemini API over OpenAI

- Decision: Use Google Gemini 2.0 Flash for AI email generation
- Rationale: Gemini API has a generous free tier (1M tokens/day via AI Studio) with no credit card required for early development. OpenAI's free tier was removed. Quality for email drafting tasks is comparable.
- Trade-off: Gemini's JSON mode is less reliable than OpenAI's. Mitigated by explicit JSON-only system prompt and fallback parsing.

ADR-004: Railway over Render / Vercel

- Decision: Deploy the Node.js API on Railway
- Rationale: Railway's free starter plan provides 500 hours/month with no sleep-on-idle behaviour (unlike Render's free tier which spins down after 15 minutes of inactivity — unacceptable for a mobile app backend).
- Trade-off: 500 hr/mo limit (~20 days of continuous uptime). For v1 with low traffic this is sufficient. Pro plan is \$5/month when needed.

ADR-005: Expo React Native over bare React Native

- Decision: Use Expo managed workflow
- Rationale: Expo provides OTA updates, EAS Build (30 free builds/month), SecureStore for JWT storage, and a unified push notification API wrapping FCM. Reduces native configuration overhead significantly for a solo or small team.
- Trade-off: Some native module limitations. Not a concern for Reilo's current feature set.